

It is worth mentioning that Starling [49] also uses a similar technique called the navigation graph to reduce the number of SSD I/O. However, GUSTANN differs from Starling in two aspects: (1) GUSTANN introduces the GPU-resident pivot graph to mitigate the load imbalance of SSDs, caused by the one-entry design. (2) GUSTANN uses GPU, instead of CPU, to perform the pivot search. Therefore, a different GPU-based search scheme is adopted, as shown above.

4.5 Discussion

4.5.1 Scalability of Data Size. In this section, we explore the possibility for scaling the data size by analyzing the storage usage of GUSTANN. GUSTANN mainly uses SSD and GPU memory, with a low DRAM usage (~ 200 MB for the bouncing buffers in the search).

On the SSD side, we store the vectors and the graph index as described in Figure 1. Billion-scale dataset (e.g., DEEP1B [52]) takes up ~ 950 GB of SSD space. Given that the capacity of modern SSDs can reach several TBs (e.g., 3.84 TB in our setup), the SSD will not be the bottleneck for scaling.

The GPU memory is mainly used for two purposes in GUSTANN. The first part is compressed vectors for distance approximation. Each compressed PQ vector is 32 bytes, and ~ 30 GB is required for the billion-scale dataset. To support larger datasets, recent quantization techniques such as LVQ [15], RabbitQ [22, 24] attempt to further reduce the memory consumption of compressed vectors (e.g., from 32 B to 16 B). Such methods could double the dataset size, and are orthogonal to GUSTANN.

The GPU memory is also used to manage the search state of the queries. GUSTANN achieves a low memory budget for the search (~ 40 KB per query) by making the deduplication procedure memory-efficient. This provides more space for larger datasets.

Another approach for scalability is to increase the GPU memory capacity. For example, with 80 GB GPU memory ($2\times$ of our experiment setup), GUSTANN could support larger datasets (i.e., 2 billion vectors) without performance loss.

Combining larger memory ($2\times$) and better quantization, GUSTANN could support 4 billion vectors with a single GPU.

4.5.2 Generalization of the Techniques. While GUSTANN targets at optimizing on-SSD ANNS with GPU, some ideas can be generalized to other problems:

- §4.2.3 provides a way to deduplicate with a low memory budget on the GPU. Other deduplication tasks on GPU could be done similarly (e.g., performing DISTINCT constraint). In the deduplication algorithm, two main techniques can be generalized: ❶ parallel predicates evaluation for on-GPU databases (Step **i** in Figure 5), and ❷ prefix sum on GPU with warp instructions (Step **ii** to **iv**).
- §4.3 introduces CPU-assisted selective transfer, which can be generalized to mitigate read amplification from SSD to GPU, potentially including on-SSD databases that utilize GPU for processing acceleration (e.g., when data processing on GPU is performed on a filtered subset of on-SSD data).

4.5.3 Limitation. To exploit the concurrent ability of GPU and SSD, GUSTANN trades off the search latency (~ 140 ms latency to reach the maximum throughput, see §5.6).

However, emerging applications, such as RAG, are not latency-critical because the latency of the whole process is mainly on the LLM side (e.g., prefill stage of LLM takes 100ms to 1s) [32, 38]. On the other hand, given the growing need for LLM and RAG applications, the ANNS in such scenarios are under much higher throughput demand, and we believe GUSTANN achieves a good throughput-latency tradeoff.

Some traditional ANNS applications, such as web search and recommendation systems, are typically considered latency-critical (e.g., 10 ms scale) [21, 30]. In these scenarios, GUSTANN may not achieve the maximum throughput due to the strict latency requirement.

5 Evaluation

In this section, we evaluate GUSTANN, seeking to answer the following questions:

- How does GUSTANN compare to other ANNS systems on both CPU and GPU for search throughput? (§5.2)
- How do designs of GUSTANN contribute to the search efficiency? (§5.3)
- Is GUSTANN sensitive to different workloads? (§5.4)
- How does GUSTANN compare to other ANNS systems in terms of cost-effectiveness? (§5.5)
- How does GUSTANN perform in terms of latency? (§5.6)

5.1 Experimental Setup

Basic configuration. We use a server with the following configuration for evaluation:

- **CPU:** 2× 28-core Intel Xeon Gold 5420+;
- **RAM:** 256 GB (8× 32 GB DDR5 4800MT/s);
- **SSD:** 6× Samsung PM1743 3.84 TB;
- **GPU:** NVIDIA A100-40GB;
- **OS:** Ubuntu 22.04 LTS with Linux kernel 6.2.0;
- **CUDA:** Version 12.3.

Datasets. We use 4 widely used public datasets for the evaluation:

- SIFT1B [31] is a dataset with 1 billion vectors of 128 `uint8` dimensions. The query set size is 10000.
- DEEP1B [52] is a dataset with 1 billion vectors of 96 `float` dimensions. The query set size is 10000.
- SIFT100M [31] is a subset of SIFT1B with 100 million vectors, and shares the same query set with SIFT1B.
- DEEP100M [52] is a subset of DEEP1B with 100 million vectors, and shares the same query set with DEEP1B.

Compared systems. We compare GUSTANN with two types of ANNS systems: (1) ANNS on CPU with SSD, and (2) ANNS on GPU with DRAM.

For ANNS on CPU with SSD, we use DiskANN, Starling, and SPANN. These systems store indexes in the SSD and use the CPU for computation. DiskANN [30] is a graph-based SSD-resident ANNS system on the CPU. Starling [49] improves DiskANN by reshuffling the indexing layout of SSDs and searching procedures. In addition, Starling uses a navigation graph to reduce the SSD I/O counts. SPANN [18] is a cluster-based ANNS system on the CPU. Cluster-based methods partition vectors into multiple clusters and search for the query in some selected clusters. SPANN leverages SSDs to store clustered vectors.

To store the vector index on the SSD, GUSTANN, DiskANN, and Starling have similar index structures and need ~65/~95 GB of storage space for SIFT100M/DEEP100M respectively. SPANN needs ~87/~235 GB for these two datasets. For the billion-scale dataset, the storage requirement is roughly 10× of the 100M-scale datasets (e.g., GUSTANN needs ~650/~950 GB for both datasets).

For ANNS on GPU with DRAM, we use BANG and RUMMY in the evaluation. BANG [48] is a recent graph-based ANNS system, and RUMMY [56] is a cluster-based ANNS system. They store indexes in DRAM and use the GPU for computation. BANG uses the DiskANN-style graph index

datasets. BANG [48] supports graph index for large-scale datasets by coordinating computation on CPU and GPU. RUMMY [56] enables cluster-based FAISS [33] to support large-scale datasets by pipelining the transfers between GPU and DRAM. These works store the index in either DRAM or GPU memory, reducing cost-effectiveness. In addition, GUSTANN optimizes the GPU-based graph search algorithm to address the limited GPU memory and achieve better concurrency.

7 Conclusion

In this paper, we propose a GPU-centric, CPU-assisted architecture for ANNS and present GUST-ANN, a billion-scale ANNS system targeted for high throughput and cost-effectiveness. Experiments show that GUSTANN achieves better throughput and is more cost-effective than current systems.

Acknowledgments

We sincerely thank all anonymous reviewers for their valuable feedback. This work is supported by Beijing Natural Science Foundation (Grant No. L242016) and the National Natural Science Foundation of China (Grant No. 62332011).

References

- [1] 2024. 4th Gen Intel® Xeon® Scalable Processors. <https://ark.intel.com/content/www/us/en/ark/products/series/228622/4th-gen-intel-xeon-scalable-processors.html>.
- [2] 2024. APP Metrics for Intel® Microprocessors. <https://www.intel.com/content/dam/support/us/en/documents/processors/APP-for-Intel-Xeon-Processors.pdf>.
- [3] 2024. Google search statistics 2024 (no. of searches per day). <https://www.demandsage.com/google-search-statistics/>.
- [4] 2024. Introducing OpenAI o1. <https://openai.com/o1/>.
- [5] 2024. New embedding models and API updates. <https://openai.com/index/new-embedding-models-and-api-updates/>.
- [6] 2024. NVIDIA A100 | NVIDIA. <https://www.nvidia.com/en-us/data-center/a100/>.
- [7] 2024. NVIDIA GPUDirect Storage. <https://docs.nvidia.com/gpudirect-storage/index.html>.
- [8] 2024. NVIDIA Tesla A100 Ampere 40 GB Graphics Processor Accelerator - PCIe 4.0 x16 - Dual Slot. <https://www.amazon.com/NVIDIA-Ampere-Graphics-Processor-Accelerator/dp/B08X13X6HF/>.
- [9] 2024. PowerEdge R760 Rack Server. https://www.dell.com/en-us/shop/dell-powered-edge-servers/new-powered-edge-r760-rack-server/spd/poweredge-r760/pe_r760_15724_vi_vp.
- [10] 2024. PowerEdge R760xa Rack Server. <https://www.dell.com/en-us/shop/cty/pdp/spd/poweredge-r760xa/>.
- [11] 2024. Samsung 32GB DDR5 4800MHz PC5-38400 ECC RDIMM 1Rx4 (EC8 10x4) Single Rank 1.1V Registered DIMM 288-Pin Server RAM Memory M321R4GA0BB0-CQK. <https://www.amazon.com/Samsung-4800MHz-PC5-38400-Registered-M321R4GA0BB0-CQK/dp/B0C35566S9>.
- [12] 2024. Samsung PM1743 3.84TB NVMe GEN5 E3.S 1T 12000MBps/12000MBps - MZ3LO3T8HCJR-00A07. <https://www.newegg.com/p/2U3-0005-000N6?Item=9SIA12KKA53814>.
- [13] 2024. Storage Performance Development Kit. <https://spdk.io>.
- [14] 2024. Supermicro 1U SuperStorage Server. https://store.supermicro.com/us_en/1u-superstorage-ssg-121e-ne316r.html.
- [15] Cecilia Aguerrebere, Ishwar Singh Bhati, Mark Hildebrand, Mariano Tepper, and Theodore Willke. 2023. Similarity Search in the Blink of an Eye with Compressed Indices. *Proc. VLDB Endow.* 16, 11 (July 2023), 3433–3446. doi:10.14778/3611479.3611537
- [16] Akhil Arora, Sakshi Sinha, Piyush Kumar, and Arnab Bhattacharya. 2018. HD-index: pushing the scalability-accuracy boundary for approximate kNN search in high-dimensional spaces. *Proc. VLDB Endow.* 11, 8 (apr 2018), 906–919. doi:10.14778/3204028.3204034
- [17] Dmitry Baranchuk, Artem Babenko, and Yury Malkov. 2018. Revisiting the inverted indices for billion-scale approximate nearest neighbors. In *Proceedings of the European Conference on Computer Vision (ECCV)*. 202–216.
- [18] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-efficient Billion-scale Approximate Nearest Neighbor Search. In *35th Conference on Neural Information Processing Systems (NeurIPS 2021)*.
- [19] Rongxin Chen, Yifan Peng, Xingda Wei, Hongrui Xie, Rong Chen, Sijie Shen, and Haibo Chen. 2024. Characterizing the Dilemma of Performance and Index Size in Billion-Scale Vector Search and Breaking It with Second-Tier Memory. *CoRR* abs/2405.03267 (2024). arXiv:2405.03267 doi:10.48550/ARXIV.2405.03267
- [20] Kenneth L. Clarkson. 1994. An Algorithm for Approximate Closest-Point Queries. In *Proceedings of the Tenth Annual Symposium on Computational Geometry, Stony Brook, New York, USA, June 6-8, 1994*, Kurt Mehlhorn (Ed.). ACM, 160–164.